Amazon Inventory Reconciliation Using AI

Aarush Patil | PES1UG24AM005
UE24CS352A Machine Learning - Mini-Project Write-up | PES University

1. Problem statement

Amazon fulfilment centres store products in fabric bins that robots carry around. Items are occasionally misplaced while being handled, so the **recorded** bin inventory no longer matches the bin's **actual** content. We predict the number of items (0-5) in a bin from a single colour photo; comparing that count with the recorded quantity flags bins that need a recount (*inventory reconciliation*). Input: bin photo. Output: item count, treated as a 6-class problem; we report accuracy, RMSE and within-one-item accuracy.

2. Dataset

We use the public **Amazon Bin Image Dataset** (ABID, 535,234 bin photos from an operating fulfilment centre; AWS Open Data bucket *aft-vbi-pds*). Each photo has a JSON file whose *EXPECTED_QUANTITY* field is the item count and serves as the label. We sample image ids at random (seed 42), keep bins with at most five items and store 30,000 images (21,000 train / 6,000 val / 3,000 test, stratified 70/20/10). Images per class (items: count) - 0: 848, 1: 3,368, 2: 6,387, 3: 7,598, 4: 6,660, 5: 5,139. Many photos are occluded by tape or bags, and some cannot be counted even by a human, which limits attainable accuracy. Images are resized to 224x224 and normalised with ImageNet statistics.

3. Approach

Classical baselines. Raw pixels (64x64x3) are standardised and compressed with randomized PCA to 256 components, then fed to logistic regression, a linear SVM, a depth-limited decision tree and an RBF-SVM (trained on an 8K subsample because kernel SVMs scale quadratically); an RBF-SVM on HOG features is also tested. Hyper-parameters are picked on the validation split.

CNN (transfer learning). ResNet34 pre-trained on ImageNet, with the final layer replaced (21.29 M parameters). Training has two stages: (i) the backbone is frozen and only the new head is trained for 2 epochs; (ii) the whole network is fine-tuned for 9 epochs with AdamW, *discriminative learning rates* (early layers lr/9, middle lr/3, head lr) and cosine annealing with warm restarts (SGDR), with the head LR (2.9e-05) chosen by a learning-rate range test. Augmentation is horizontal flip and mild colour jitter; at test time the softmax outputs of the image and its mirror image are averaged (TTA).

Ordinal head (ablation). Counts are ordered, so a miss by one item should cost less than a miss by three. Besides the standard 6-way softmax with cross-entropy we train a head with five cumulative binary outputs P(count>k), k=0..4, trained with binary cross-entropy; the prediction is the number of outputs above 0.5.

Reconciliation. The public data only has true counts, so recorded-inventory errors are *simulated*: each test bin's recorded count is wrong by +-1 or +-2 items with probability 15%. A bin is flagged when the predicted count differs from the recorded one; 1-P(recorded count) is also evaluated as a threshold-free variance score (AUROC), averaged over 20 random draws.

4. Implementation overview

Everything is in a private GitHub repository with a README and a one-click Colab notebook: *download_data.py* (HTTPS download, resize, resumable), *build_splits.py*, *baselines.py* (scikit-learn), *train_cnn.py* (PyTorch/torchvision: two-stage fine-tuning, SGDR, LR finder, mixed precision, TTA), *make_results.py* (metrics, figures, variance simulation), *demo.py* (Gradio app and CLI). The best CNN trained in 15 min on Tesla T4 and takes 6.2 ms per image at batch size 1 on that device.

5. Results

Table 1 lists test-set results (model selection used the validation split; * = best CNN by validation accuracy).

Model	Test acc.	RMSE	Within 1
Random guess	16.7%	-	-
Majority class	25.3%	1.35	68.8%
Logistic regression (PCA)	28.5%	1.37	73.1%
Linear SVM (PCA)	27.7%	1.37	73.0%
Decision tree (PCA)	25.5%	1.57	65.7%
RBF-SVM (PCA)	29.8%	1.32	74.9%
RBF-SVM (HOG)	31.9%	1.32	75.3%
ResNet34, softmax-head + TTA *	41.2%	1.11	83.4%
ResNet34, ordinal-head + TTA	39.4%	1.04	86.6%

Table 1: test results on the held-out split.

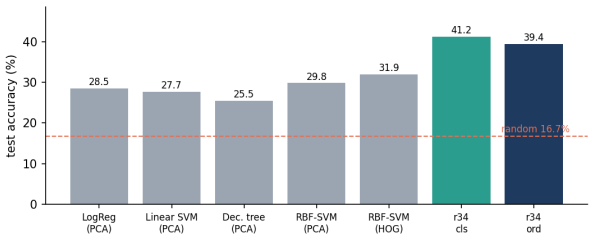


Fig. 1: test accuracy of all models (dashed line = random guessing).

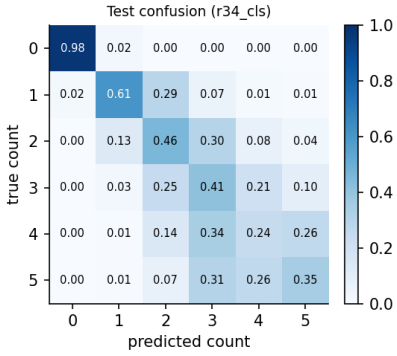


Fig. 2: row-normalised confusion matrix of the best CNN on the test set.

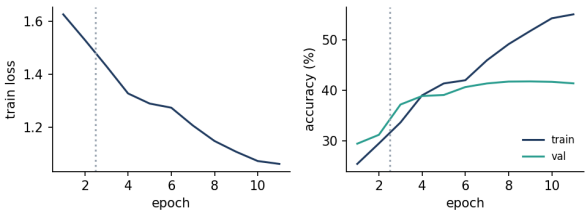


Fig. 3: training loss and accuracy of the best CNN (dotted line = start of full fine-tuning).

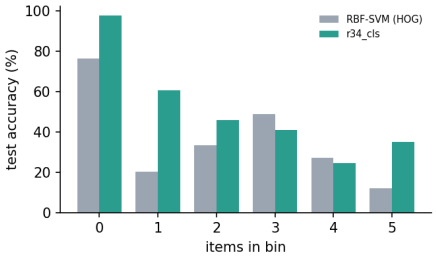


Fig. 4: per-class test accuracy, best baseline vs best CNN.

6. Conclusions

The fine-tuned ResNet34 (softmax-head) reaches **41.2%** test accuracy (RMSE 1.11, 83.4% within one item), i.e. 2.5x random guessing and +29% relative to the best classical model (RBF-SVM (HOG), 31.9%). Learned convolutional features clearly beat the hand-crafted/PCA features on this non-linear, occlusion-heavy task.

Test-time augmentation changed accuracy by +0.5 points.

The ordinal head improved RMSE (1.04 vs 1.11 for softmax) at 39.4% vs 41.2% accuracy.

Accuracy is highest for bins with 0 items (97.6%) and lowest for 4 items (24.5%); most errors are off by exactly one item.

With 15% simulated recording errors, flagging bins whose predicted count differs from the recorded one gives precision 19.4%, recall 81.4% (F1 0.31); the probability score reaches AUROC 0.73.

For context, the published Stanford ResNet34 result (56.2% / RMSE 0.90, larger and different subset) is below ours (-15.0 points); the setups differ, so this is only indicative.

Limitations / future work. Occluded bins cap accuracy (cleaning them or using several camera angles may help), results are from one split and seed, the 224-px resize may hide small items, and recording errors were simulated. Next steps: ensembling, higher-resolution inputs, and evaluating on real inventory-variance logs.